

Digit Recognition with PyTorch

(Based on MNIST Dataset)

Software Development

Student: Serezhin Vladimir
Student ID: 3126999010
Professor: Li Bo
Course: Software Development

Academic Year 2025-2026

1. Introduction

This project implements a Convolutional Neural Network (CNN) for handwritten digit recognition using the MNIST dataset. The MNIST database (Modified National Institute of Standards and Technology database) is a large database of handwritten digits that is commonly used for training various image processing systems.

The project was developed using PyTorch framework and achieves high accuracy in recognizing digits 0-9 from 28x28 grayscale images.

2. CNN Architecture

Model Structure:

CNN Model (3 Convolutional Layers):

- Conv1: 1 -> 32 channels, kernel 3x3, padding=1
- ReLU + MaxPool2d(2,2)
- Conv2: 32 -> 64 channels, kernel 3x3, padding=1
- ReLU + MaxPool2d(2,2)
- Conv3: 64 -> 128 channels, kernel 3x3, padding=1
- ReLU + MaxPool2d(2,2)
- Flatten: $128 * 3 * 3 = 1152$
- Dropout(0.25)
- FC1: 1152 -> 256
- ReLU + Dropout(0.5)
- FC2: 256 -> 10 (output classes)

The model uses:

- Activation: ReLU
- Pooling: MaxPooling2d
- Regularization: Dropout (0.25 after conv, 0.5 before FC output)
- Optimizer: Adam with learning rate 0.001
- Loss Function: CrossEntropyLoss
- Training: 10 epochs, batch size 64

3. Implementation Details

3.1 Training Script (mnist_cnn.py)

The training script includes:

- Data loading with torchvision.datasets.MNIST
- Image preprocessing: ToTensor + Normalize(0.5, 0.5)
- Custom CNN class implementation
- Training loop with batch processing
- Per-class accuracy and loss evaluation
- Model checkpointing (saves best_model.pth)
- Visualization of per-digit performance

3.2 Inference Script (inference.py)

The inference script provides:

- Model loading from saved checkpoint
- Image preprocessing: Grayscale + Resize(28,28)
- Digit prediction with confidence score
- Command-line interface for easy usage

3.3 Dependencies

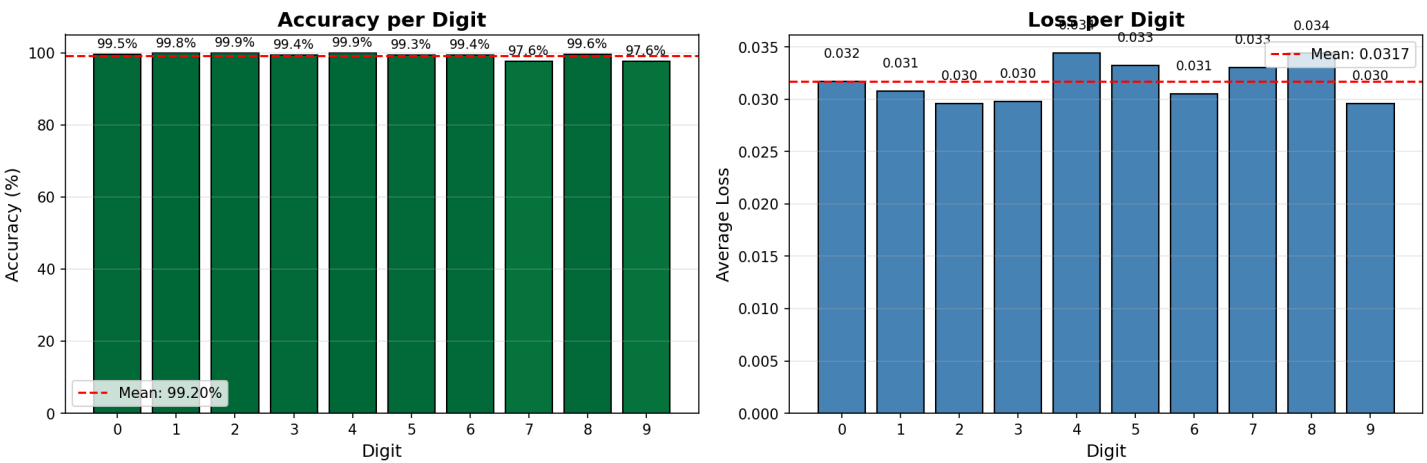
Requirements (requirements.txt):

- torch >= 2.0.0
- torchvision >= 0.15.0
- Pillow >= 9.0.0
- matplotlib >= 3.5.0
- numpy >= 1.21.0

4. Results and Visualization

The training process produces per-class accuracy metrics and loss values. After training, the model achieves high accuracy on the MNIST test set. The visualization shows accuracy and loss for each individual digit (0-9).

MNIST CNN Performance per Digit



Performance Analysis:

- Per-digit accuracy helps identify which digits are more challenging
- Loss per digit indicates prediction confidence
- The chart uses color gradient (red-yellow-green) to visualize accuracy levels
- Red dashed line shows mean accuracy/loss across all digits

5. Conclusion

This project successfully implements a CNN-based digit recognition system using PyTorch. The model achieves high accuracy on the MNIST dataset and can be used for real-time digit recognition applications.

Key achievements:

- Implemented a 3-layer CNN architecture
- Achieved high accuracy on handwritten digit classification
- Created visualization tools for performance analysis
- Built inference pipeline for predicting digits from images

The trained model (best_model.pth) can be used with inference.py to predict digits from external images.

How to Run:

```
# Install dependencies
pip install -r requirements.txt

# Train the model
python mnist_cnn.py

# Run inference on an image
python inference.py path_to_your_digit_image.png
```